

NOVACART

End-to-End SQL Database Lab

From Business Requirements to Database Analysis

Platform: Microsoft SQL Server / SSMS

Student: _____

Section: _____

ID: _____

Student Assignment — No Solutions Included

Build the database, populate it with meaningful data, and use SQL to answer business questions.

1. Lab Overview

NovaCart is a growing online retail company that sells electronics, fashion items, home appliances, books, and accessories through its online platform. The company needs a relational database to manage customers, products, orders, payments, and customer reviews efficiently.

This lab takes you through a complete database workflow: understand the business requirements, design the database, implement it in SQL Server, populate it with data, and use SQL to answer business questions.

Important: You are expected to build the database yourself. The student package does not provide a ready-made DDL script, completed ERD, completed relational mapping, or SQL solutions.

2. Learning Objectives

- Translate a business description into entities, attributes, relationships, cardinalities, and participation constraints.
- Design a conceptual ERD and map it to a relational model.
- Implement a relational database using SQL Server DDL.
- Use primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints appropriately.
- Create a meaningful dataset that supports realistic SQL analysis.
- Write SQL using filtering, sorting, aggregation, GROUP BY, HAVING, joins, subqueries, CTEs, window functions, and views.
- Explain important database design decisions and SQL choices.

3. Business Case

NovaCart is a growing online retail company. Customers must create an account before making purchases. For each customer, the company stores the customer's full name, email address, phone number, home address, and the date they joined the platform. Each customer has a unique account and may place multiple orders over time.

The company maintains a catalog of products. For every product, the system stores the product name, category, selling price, and available stock quantity. Products are grouped into categories to simplify browsing.

Customers can place orders through the website. For every order, the system records the order date and its current status. Valid order statuses are Pending, Shipped, Delivered, and Cancelled. An order can contain multiple products, and the same product can appear in many different orders. The quantity purchased must be recorded for each product within an order.

Product prices may change over time. Therefore, the system must also store the unit price that applied when the product was included in an order. Historical order prices must not depend on a product's current catalog price.

Each order is associated with a single payment. For every payment, the company records the payment date, payment amount, and payment method. Valid payment methods are Credit Card, PayPal, and Cash on Delivery (COD).

After receiving their orders, customers may leave reviews for products they have purchased. Each review contains a rating from 1 to 5, an optional comment, and the date the review was submitted. A customer may review multiple products, and each product may receive reviews from many different customers.

4. Business Rules to Consider

- A customer must exist before an order can belong to that customer.
- An order belongs to one customer; a customer may place multiple orders.
- An order can contain multiple products; a product can appear in multiple orders.
- Quantity is associated with a product within a specific order.
- The unit price stored for an order line represents the price at the time of purchase.
- Each order has one associated payment.
- A payment uses one supported payment method.
- A review belongs to one customer and one product.
- A review rating must be between 1 and 5.
- A customer should review a product only after purchasing it; consider whether and how this rule can be enforced by the database.

5. Phase 1 — Database Design

Do not start by writing CREATE TABLE statements. First analyze the business case and produce the conceptual design.

5.1 Requirements Analysis

Identify the entities, important attributes, relationships, cardinalities, and participation constraints implied by the business case.

5.2 Conceptual ERD

Create a conceptual ERD for NovaCart. Focus on entities, attributes, and relationships. Do not place implementation-level foreign keys in the conceptual ERD.

5.3 Relational Mapping

Map your conceptual ERD into a relational model. Show every table, primary key, and foreign key clearly. Make sure the many-to-many relationship between orders and products is represented correctly.

6. Design Reasoning Questions

Answer briefly in your own words. These questions test database reasoning, not SQL syntax.

1. Why is an intermediate table needed between Orders and Products?
2. Why should quantity be stored for an order-product combination rather than in Products?
3. Why should unit_price be stored in order details instead of relying only on the current Products.price?
4. Why should order status be restricted to the four business-defined values?
5. Why is order_id required to be unique in Payments if each order has exactly one payment?
6. Which business rules can be enforced directly with common relational constraints, and which review rule may require additional logic?

7. Phase 2 — Database Implementation

Create a database named NovaCartDB in Microsoft SQL Server and implement your relational design using DDL.

Your DDL must include appropriate database and table definitions, primary keys, foreign keys, NOT NULL constraints, UNIQUE constraints, CHECK constraints, suitable SQL Server data types, and identity columns where appropriate.

Do not change the business requirements simply to make the implementation easier.

8. Phase 3 — Data Population

Insert at least 10 records into every table. Your dataset must be intentionally designed so the SQL missions can be meaningfully tested.

Your dataset must satisfy all of these conditions:

- At least one customer has more than one order.
- At least one customer has no orders.
- At least one order contains more than one product.
- At least one product appears in more than one order.

- At least one product has never been ordered.
- At least one product has never received a review.
- Orders contain multiple statuses, including at least one Delivered and at least one non-Delivered order.
- All supported payment methods are represented.
- Reviews contain different rating values, including ratings from both the lower and upper parts of the 1-5 range.
- At least one customer has more than one review or more than one reviewed product.
- Dates and amounts vary enough to make grouping, ordering, monthly analysis, and ranking meaningful.

Do not add artificial rows solely to make a query return a result. Design the dataset as if it represented a small but realistic online store.

9. Phase 4 — SQL Missions

Complete the missions using the database you designed and populated. Use meaningful aliases and readable SQL formatting.

Mission 1 — Customer & Product Overview

1. Display all customers.
2. Display the product name, category, and current price for every product.
3. Display products whose current price is greater than 5000.
4. Display all customers ordered by join date, newest first.
5. Display the total number of customers.

Mission 2 — Aggregation & Business Totals

1. Calculate the average current product price.
2. Display the highest and lowest current product prices.
3. Calculate the total available stock quantity.
4. Calculate the total amount recorded in Payments.
5. Display the number of orders for each order status.
6. Display the total payment amount for each payment method.

Mission 3 — Order & Sales Analysis

1. Calculate the total sales amount for each order using quantity multiplied by the historical unit price.
2. Display only orders whose total sales exceed 5000.
3. Display each order together with the customer's name, order date, and status.
4. Display each order with its products, purchased quantities, and historical unit prices.
5. Display the total amount spent by each customer.

Mission 4 — Reviews & Relationships

1. Display the number of reviews received by each product, including products with no reviews.
2. Display all reviews together with the customer name and product name.
3. Display customers who have placed at least one order.
4. Display products that have never been ordered.
5. Display products that have never received a review.
6. Display all customers and their number of orders, including customers who have never placed an order.

Mission 5 — Subqueries

1. Display customers who placed more orders than the average number of orders among customers who placed at least one order.
2. Display products whose current price is above the average current product price.
3. Display customers whose total spending exceeds the average among customers who made at least one payment.

Mission 6 — Common Table Expressions

1. Using a CTE, calculate total revenue by month.

2. Using a CTE, calculate total spending by customer and return only customers whose spending exceeds 10000.

Mission 7 — Window Functions

1. Rank customers by total spending using RANK(), highest spending first.
2. Rank products by total quantity sold using DENSE_RANK(), highest quantity first.
3. Display each payment together with the previous payment amount using LAG(), ordered by payment date and payment ID.
4. Display a running total of payment amounts ordered by payment date and payment ID.

Mission 8 — Views

1. Create a view named vw_revenue_by_month that displays monthly revenue.
2. Create a view named vw_best_selling_products that displays product name, total quantity sold, and total revenue.
3. Create a view named vw_customer_summary that displays customer name, number of orders, and total amount spent.

10. SQL Reasoning & Quality Checks

Before considering your work complete, review your queries and database design using the questions below.

1. Which of your queries require GROUP BY? Why?
2. Which queries require HAVING rather than WHERE? Explain the difference using one of your solutions.
3. Which questions require a LEFT JOIN because rows with no matching records must still appear?
4. Where could duplicate rows be introduced by a join? How did you make sure your aggregation remains correct?
5. Why should revenue calculations for an order use OrderDetails.unit_price rather than Products.price?
6. Why are both RANK() and DENSE_RANK() included in this lab? Describe the difference when ties occur.

11. Submission Requirements

Submit the following files with the exact names below:

File	Required content
1_ERD.png	Your conceptual ERD.
1_Mapping.png	Your ERD-to-relational mapping.
2_NovaCart_DDL.sql	Your complete SQL Server DDL.
3_NovaCart_Insert_Data.sql	Your INSERT statements and required dataset.
4_NovaCart_Answers.sql	Your SQL solutions for all assigned questions.

12. Submission Rules

- Use Microsoft SQL Server / SSMS.
- Do not submit a ready-made database file instead of the required scripts.
- The DDL script must execute successfully from a clean environment.
- The INSERT script must execute successfully against your own DDL.
- All foreign keys must reference valid records.
- All required constraints must be implemented in the database design.
- Every SQL question should execute without errors on your completed database.
- Use meaningful SQL formatting and naming conventions.
- Keep table and column names consistent across all deliverables.
- Use the separate Hints PDF only when you need guidance; it intentionally does not contain complete solutions.

13. Final Student Checklist

Before submission, verify:

- The ERD represents the business requirements and contains no implementation-only foreign keys.

- The relational mapping contains all required primary keys and foreign keys.
- The DDL creates NovaCartDB and all required tables successfully.
- Primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints are implemented where appropriate.
- Every table contains at least 10 records.
- The dataset satisfies all required data scenarios.
- The SQL questions were solved against your own database.
- Queries return meaningful results rather than relying on accidental or hard-coded values.
- All required files use the specified names.

End of Student Assignment

Start with the business requirements. Design first, implement second, and query last.